

Vibrational Wiring Deduction Transformer:

A Generative Framework for
Graph-Structured Latent Spaces and Spectral Associative Memory

Lorenzo Moriondo

Independent Researcher — tuned.org.uk

ORCID: 0000-0002-8804-2963

[github repo](#)

June 2026

Abstract

We present the *Wiring Autoencoder v2* (WAE v2), a fully Bayesian generative model that upgrades the original WAE family with three coupled spectral priors derived from probabilistic PCA in a Laplacian eigenbasis (Spectral-PPCA). The v1 architecture used a hard high-frequency energy penalty (J_{freq}) and an isotropic Gaussian latent prior. WAE v2 replaces both with principled variational KL terms: (i) a spectral-basis KL over the loading matrix $\mathbf{S}$ weighted by Laplacian eigenvalues, implementing a *spectral Occam's razor*; (ii) a closed-form Gamma/Exponential KL over per-mode weights $\boldsymbol{\omega}$, providing soft τ -mode band-limiting; and (iii) the standard isotropic Gaussian latent KL retained for stability (the Laplacian-precision latent KL is preserved as a design option and ablation, but is not included in the default three-term ELBO following architectural review in PR #35). After training, a *spectral artefact* $\mathcal{A}(\mathcal{I})$ is extracted and used to initialise a transformer with pre-built spectral associative memory. The two-phase (offline training, online inference) architecture separates spectral learning from deployment, with frozen eigenpairs eliminating all runtime $O(N^3)$ eigendecompositions during training. We detail the complete module design, three-term ELBO, stability analysis, benchmark metrics, and associative memory construction.

Contents

1	Introduction	3
1.1	Contributions	3
2	Background	3
2.1	Probabilistic PCA	3
2.2	Graph Laplacian Spectral Theory	4
2.3	Wiring Autoencoder (WAE)	4
3	Spectral-PPCA Generative Framework	4
3.1	Three Structural Priors	4
4	WAE v2: Three-Term ELBO	5
4.1	Variational Family	5
4.2	ELBO Decomposition	5
4.3	Closed-Form KL Terms	5
4.4	Comparison with WAE v1	6

5	Model Architecture	6
5.1	Data Flow	6
5.2	WiringEncoderV2	6
5.3	SpectralLoadingDecoder	7
5.4	DiffusionDecoder (Unchanged)	7
5.5	WiringAutoencoderV2	7
6	Spectral Artefact and Associative Memory	8
6.1	Spectral Artefact	8
6.2	SpectralAssociativeMemory	8
7	Two-Phase Architecture	8
8	Stability Analysis	9
9	Benchmark Metrics	9
10	Algorithm Tracks	10
11	Relationship to Prior Work	10
12	Discussion and Open Problems	11
13	Conclusion	11

1 Introduction

Graph-structured latent variable models occupy an increasingly important niche between classical variational autoencoders (VAEs) and geometric deep learning: they can encode both the *content* of nodes and the *relational structure* that organises them. The original Wiring Autoencoder [?] recognised this, using a differentiable graph Laplacian $L(z)$ as an intermediate latent object whose spectral properties are regularised by a hard frequency penalty $J_{\text{freq}} = \sum_{j>k} \lambda_j(L(z))$. While effective in practice, this penalty is a heuristic rather than a proper probabilistic prior, which makes it difficult to:

1. compare competing ArrowSpace indices $\mathcal{I}_1, \mathcal{I}_2$ via principled model selection;
2. interpret the learned spectral structure as a posterior over loading matrices $\mathbf{W}$;
3. extract a reusable *spectral artefact* whose keys are approximately orthonormal and whose values are grounded in the data manifold.

WAE v2 addresses all three limitations by embedding the Wiring Autoencoder within the Spectral-PPCA framework [Moriondo, 2026a], a Bayesian generative model in which the loading matrix $\mathbf{W} = \mathbf{U}_{1:q} \text{diag}(\boldsymbol{\omega}) \mathbf{S}$ lives explicitly in the Laplacian eigenbasis and is endowed with eigenvalue-weighted shrinkage priors.

1.1 Contributions

- A three-term ELBO replacing J_{freq} and preserving the isotropic latent prior (Section 4).
- A `SpectralLoadingDecoder` that maps latent codes to eigenbasis-factorised loading matrices (Section 5).
- A `WiringEncoderV2` with VDT recurrence and `ModeWeightHead` producing variational Gamma parameters (Section 5).
- A `SpectralAssociativeMemory` module that pre-builds a Hopfield memory from the spectral artefact (Section 6).
- A two-phase offline/online architecture eliminating runtime eigensolves during training (Section 7).
- An eight-metric evaluation suite including ELBO Bayes factors for ArrowSpace index selection (Section 9).

2 Background

2.1 Probabilistic PCA

Standard PPCA [Tipping and Bishop, 1999] defines a linear-Gaussian generative model:

$$\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_q), \quad \mathbf{x} \mid \mathbf{z} \sim \mathcal{N}(\mathbf{W}\mathbf{z} + \boldsymbol{\mu}, \sigma^2 \mathbf{I}_d), \quad (1)$$

producing marginal $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \mathbf{W}\mathbf{W}^\top + \sigma^2 \mathbf{I}_d)$. Maximum-likelihood estimation recovers the PCA loading matrix in the column space of the q largest singular vectors of the data matrix. The fully Bayesian extension places priors over $\mathbf{W}$, $\mathbf{z}$, and σ^2 , enabling model selection over the latent dimensionality q via the marginal likelihood [Minka, 2001].

2.2 Graph Laplacian Spectral Theory

Let $G = (V, E, w)$ be a weighted graph with node set $|V| = N$. The normalised graph Laplacian is $L = D^{-1/2}(D - W)D^{-1/2}$ where $D_{ii} = \sum_j w_{ij}$. Eigendecomposition $L = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$ yields eigenvectors $\mathbf{U} = (\mathbf{u}_1, \dots, \mathbf{u}_N) \in \mathbb{R}^{N \times N}$ and non-negative eigenvalues $0 \leq \lambda_1 \leq \dots \leq \lambda_N$. Low-frequency eigenvectors (small λ_k) are smooth over G ; high-frequency ones (large λ_k) oscillate rapidly across edges [Chung, 1997].

Given ArrowSpace index $\mathcal{I}$, we denote the frozen q -truncated eigendecomposition

$$L(\mathcal{I}) = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top, \quad \mathbf{U}_q = \mathbf{U}_{:,1:q} \in \mathbb{R}^{N \times q}, \quad \mathbf{\Lambda}_q = \text{diag}(\lambda_1, \dots, \lambda_q). \quad (2)$$

These eigenpairs are computed once offline and used as frozen constants throughout training.

2.3 Wiring Autoencoder (WAE)

WAE v1 trains a differentiable Laplacian $L(\mathbf{z})$ whose topology is fixed (kNN + RBF graph) but whose edge weights are learned by a mixture-of-experts decoder $W_{\text{dec}} : \mathbf{z} \mapsto \delta_e$. The training objective is

$$\mathcal{L}_{\text{WAE}} = \mathbb{E}_q[\log p(\mathbf{x} \mid \mathbf{z})] - \beta \cdot \text{KL}(q(\mathbf{z}) \parallel \mathcal{N}(\mathbf{0}, \mathbf{I})) - \alpha \cdot J_{\text{freq}}(L(\mathbf{z})), \quad (3)$$

where $J_{\text{freq}} = \sum_{j>k} \lambda_j(L(\mathbf{z}))$ penalises high-frequency wiring. The reconstruction is provided by **TauModeDiffusion**: the heat kernel $K_\tau = \mathbf{U}_k \exp(-t\mathbf{\Lambda}_k)\mathbf{U}_k^\top$ applied to an embedding table $\mathbf{E}$, with learnable diffusion time t .

Three limitations of this design motivate WAE v2: (a) J_{freq} is a deterministic penalty with no prior interpretation; (b) the latent prior is isotropic and ignores the graph geometry; (c) there is no principled post-training artefact for downstream use.

3 Spectral-PPCA Generative Framework

Following Moriondo [2026a], we define the Spectral-PPCA prior by replacing the standard PPCA loading matrix with one that lives in the Laplacian eigenbasis.

3.1 Three Structural Priors

Prior 1: Spectral-basis loading. Parametrise $\mathbf{W} = \mathbf{U}_{1:q} \text{diag}(\boldsymbol{\omega}) \mathbf{S}$ where $\mathbf{U}_{1:q} \in \mathbb{R}^{d \times q}$ are the q lowest-frequency eigenvectors of $L(\mathcal{I})$ (frozen) and $\mathbf{S} \in \mathbb{R}^{q \times q}$ lives in the spectral basis. Place an eigenvalue-weighted Gaussian shrinkage prior on $\mathbf{S}$:

$$p(\mathbf{S} \mid \mathcal{I}) \propto \exp\left(-\frac{\lambda_s}{2} \text{tr}(\mathbf{S}^\top \mathbf{\Lambda}_q \mathbf{S})\right). \quad (4)$$

This implements a *spectral Occam's razor*: high-frequency components (large λ_k) are penalised exponentially more strongly than low-frequency ones, pulling the loading matrix toward smooth directions of the index graph.

Prior 2: τ -mode frequency prior. Let $\omega_k > 0$ be a weight on the k -th mode. The τ -mode prior is a product of exponentials with rates that scale with Laplacian eigenvalues:

$$p(\omega_k \mid \tau, \lambda_k) = \text{Exp}(\tau \lambda_k), \quad k = 1, \dots, q. \quad (5)$$

Small λ_k (low-frequency modes) receive weak penalisation; large λ_k receive strong penalisation, naturally selecting the effective spectral bandwidth. The loading matrix is a mode-weighted linear combination $\mathbf{W} = \mathbf{U}_{1:q} \text{diag}(\boldsymbol{\omega}) \mathbf{S}$.

Prior 3 (design option): Laplacian-precision latent prior. For a richer latent geometry one can replace the isotropic prior with a Laplacian-precision prior:

$$p(\mathbf{z} \mid \mathcal{I}) = \mathcal{N}(\mathbf{0}, (\mathbf{I} + \beta L_s)^{-1}), \quad (6)$$

where L_s is a sample-graph Laplacian built from the current batch of latent means (stop-gradient). This enforces smoothness of latent codes over the data manifold. The resulting KL expands to

$$\text{KL}(q(\mathbf{z}) \parallel p_{\text{Lap}}(\mathbf{z})) = \frac{1}{2} \left[\text{tr}(\Sigma_z(\mathbf{I} + \beta L_s)) + \boldsymbol{\mu}_z^\top (\mathbf{I} + \beta L_s) \boldsymbol{\mu}_z - d - \log \det \Sigma_z + \log \det (\mathbf{I} + \beta L_s)^{-1} \right], \quad (7)$$

where the cross-term $\boldsymbol{\mu}_z^\top L_s \boldsymbol{\mu}_z$ is the graph Dirichlet energy of the latent means. *This prior is preserved in the WAE v2 codebase as an ablation option (`use_isotropic_kl=False` in `WiringEncoderV2`) but is **not** included in the default three-term ELBO following architectural review in PR #35.*

4 WAE v2: Three-Term ELBO

4.1 Variational Family

We adopt a mean-field amortised variational family:

$$q_\phi(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}_z, \text{diag}(\exp(\mathbf{s}_z))), \quad (8)$$

$$q_\phi(\mathbf{S} \mid \mathbf{x}) = \mathcal{N}(M_S, \text{diag}(\exp(\mathbf{v}_S))), \quad (9)$$

$$q_\phi(\omega_k \mid \mathbf{x}) = \text{Gamma}(a_k, b_k), \quad (10)$$

where all parameters are outputs of the encoder network `WiringEncoderV2`. The Gamma distribution is reparametrisable via the implicit reparametrisation trick [Figurnov et al., 2018]; in practice we parametrise through $(\log a_k, \log b_k)$ from a `ModeWeightHead` linear layer.

4.2 ELBO Decomposition

The default WAE v2 training objective is the following three-term ELBO (minimisation form, as implemented in `wae/model.py`):

$$\mathcal{L}_{\text{WAE-v2}} = \underbrace{\mathbb{E}_q[\|\mathbf{x} - \hat{\mathbf{x}}\|^2 / 2\sigma^2]}_{\text{reconstruction}} + \underbrace{\text{KL}(q(\mathbf{z}) \parallel \mathcal{N}(\mathbf{0}, \mathbf{I}))}_{\text{isotropic latent KL (kl_z)}} + \underbrace{\text{KL}(q(\mathbf{S}) \parallel p(\mathbf{S} \mid \mathcal{I}))}_{\text{spectral basis KL (kl_S)}} + \underbrace{\text{KL}(q(\boldsymbol{\omega}) \parallel p(\boldsymbol{\omega} \mid \tau, \boldsymbol{\Lambda}))}_{\tau\text{-mode KL (kl_tau)}} \quad (11)$$

All four terms are non-negative; the total loss is minimised (ELBO is maximised by negating) during gradient descent.

4.3 Closed-Form KL Terms

Isotropic latent KL.

$$\text{KL}(q(\mathbf{z}) \parallel \mathcal{N}(\mathbf{0}, \mathbf{I})) = -\frac{1}{2} \sum_{i=1}^{d_z} (1 + s_{z,i} - \mu_{z,i}^2 - e^{s_{z,i}}). \quad (12)$$

Spectral-basis KL. For the entry (k, j) of $\mathbf{S}$, with precision $\lambda_s \lambda_k$ (eq. (4)):

$$\text{KL}_{kj} = \frac{1}{2} [\lambda_s \lambda_k (v_{S,kj} + S_{kj}^2) - v_{S,kj} - 1 - \log(\lambda_s \lambda_k)], \quad (13)$$

where $v_{S,kj} = \exp(\log \text{var}_{S,kj})$. In the current implementation, the log-variance is approximated as $\log \text{var}_{S,kj} = \log(S_{kj}^2 + \epsilon)$, deriving uncertainty directly from the magnitude of the mean — a simplification that avoids adding a separate posterior variance head to `SpectralLoadingDecoder`. Future work should add a dedicated variance head for a proper amortised posterior.

τ -mode KL. For mode k with $q(\omega_k) = \text{Gamma}(a_k, b_k)$ and $p(\omega_k) = \text{Gamma}(1, \tau\lambda_k)$, the closed-form KL is:

$$\text{KL}(\text{Gamma}(a, b) \parallel \text{Exp}(r)) = \log b - \log r + \ln \Gamma(a) + (1 - a)\psi(a) + \frac{ab}{r}, \quad r = \tau\lambda_k, \quad (14)$$

where ψ is the digamma function. This is summed over all q modes and averaged over the batch.

4.4 Comparison with WAE v1

Table 1: ELBO comparison: WAE v1 vs WAE v2.

Term	WAE v1	WAE v2	Note
Reconstruction	$-\ \mathbf{x} - \hat{\mathbf{x}}\ ^2 / 2\sigma^2$	Same	σ learnable
Latent KL	$\text{KL}(q(\mathbf{z}) \parallel \mathcal{N}(\mathbf{0}, \mathbf{I}))$	Same (isotropic)	PR #35 decision
Spectral basis KL	—	$\text{KL}(q(\mathbf{S}) \parallel p(\mathbf{S} \mid \mathcal{I}))$	eigenvalue-weighted Gaussian
τ -mode term	αJ_{freq} (hard)	$\text{KL}(q(\boldsymbol{\omega}) \parallel p(\boldsymbol{\omega} \mid \tau, \boldsymbol{\Lambda}))$	variational Gamma prior
Lap. precision latent KL	—	<i>design option</i>	ablation only

5 Model Architecture

5.1 Data Flow

Figure 1 illustrates the end-to-end data flow.

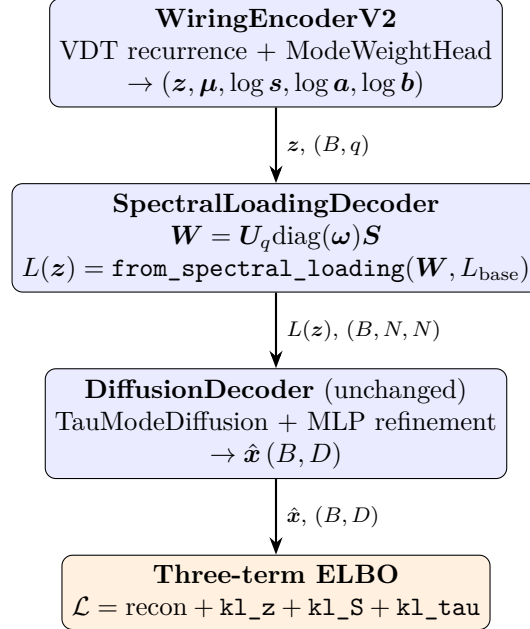


Figure 1: WAE v2 data flow. The eigenpairs $(\mathbf{U}_q, \boldsymbol{\Lambda}_q)$ of $L(\mathcal{I})$ are frozen constants passed to both encoder and decoder; no Laplacian is computed at runtime.

5.2 WiringEncoderV2

WiringEncoderV2 (`wae/encoder.py`) replaces the v1 MLP encoder with a VDT (Vibrational Discrete Transformer) recurrence stack:

1. **Input projection:** linear layer mapping $(B, D) \rightarrow (B, N \cdot d)$, reshaped to (B, N, d) .
2. **VDT blocks:** K `VibrationalStateBlock` layers operating on (B, N, d) state matrices using the feature-space Laplacian L_f . The recurrence implements a discrete damped wave equation in the graph spectral domain.
3. **Modal projection:** mean-pool output over the node dimension after projecting through eigenvectors $\mathbf{v} \in \mathbb{R}^d$.
4. **Readout heads:** separate linear layers producing $\boldsymbol{\mu}, \log \mathbf{s}$ (for $q(\mathbf{z})$) and `ModeWeightHead` producing $(\log \mathbf{a}, \log \mathbf{b})$ for $q(\boldsymbol{\omega})$.

The optional λ -fingerprint (ArrowSpace spectral histogram of $L(\mathcal{I})$) can be concatenated to $\mathbf{x}$ before the input projection (`use_lambda_features=True`).

ModeWeightHead. A single linear layer `Linear(feats_dim, 2q)` splits into $(\log \mathbf{a}, \log \mathbf{b}) \in \mathbb{R}^q \times \mathbb{R}^q$. The Gamma parameters are obtained as $a_k = e^{\log a_k} > 0$, $b_k = e^{\log b_k} > 0$. The mode-weight KL (14) is computed from these in `wae/spectral.py:tau_mode_kl`.

5.3 SpectralLoadingDecoder

`SpectralLoadingDecoder` (`wae/wiring_decoder.py`) replaces `WiringDecoder` as the default v2 wiring decoder. The forward pass computes:

$$\mathbf{S} = \mathbf{S_net}(\mathbf{z}).\text{view}(B, q, q), \quad (15)$$

$$\boldsymbol{\omega} = \exp(\boldsymbol{\omega_net}(\mathbf{z})), \quad (16)$$

$$\mathbf{W} = \mathbf{U}_q^\top \otimes (\boldsymbol{\omega}_k \mathbf{S}), \quad \text{i.e.} \quad \mathbf{W} = \mathbf{U}_q.\text{unsqueeze}(0) \otimes (\boldsymbol{\omega}.\text{unsqueeze}(-1) * \mathbf{S}), \quad (17)$$

$$L(\mathbf{z}) = \text{DifferentiableLaplacian.from_spectral_loading}(\mathbf{W}, L_{\text{base}}). \quad (18)$$

Edge weights in (18) are synthesised as $w_{ij} = w_{ij}^{\text{base}} \cdot \sigma(-\|\mathbf{W}_i - \mathbf{W}_j\|^2)$ where $\mathbf{W}_i$ is the i -th row of $\mathbf{W}$, and σ is the logistic sigmoid. This path is fully differentiable from $L(\mathbf{z})$ back to $\mathbf{z}$ through $\mathbf{W}$.

Initialisation. `S_net.weight` is initialised so the initial $\mathbf{S}_b \approx \mathbf{I}_q$ (near-identity); `omega_net` is initialised near zero so initial $\omega_k \approx 1$ for all modes. This gives a warm-start close to the eigenbasis reparametrisation $\mathbf{W} \approx \mathbf{U}_q$.

Config dispatch. The v1 `WiringDecoder` (mixture-of-experts) is retained as a fallback: `decoder_type: spectral` selects `SpectralLoadingDecoder`; `decoder_type: mixture_of_experts` selects `WiringDecoder`.

5.4 DiffusionDecoder (Unchanged)

`DiffusionDecoder` is identical to v1. It wraps `TauModeDiffusion`: the truncated heat kernel

$$K_\tau = \mathbf{U}_k \text{diag}(e^{-t\boldsymbol{\Lambda}_k}) \mathbf{U}_k^\top, \quad (19)$$

with learnable log-diffusion-time $\log t$. A small MLP refinement residually corrects the output. Reconstruction loss is Gaussian NLL: $\|\mathbf{x} - \hat{\mathbf{x}}\|^2 / (2\sigma^2)$.

5.5 WiringAutoencoderV2

`WiringAutoencoderV2` (`wae/model.py`) assembles all three modules. Its `forward()` method returns a dict with exactly nine keys: `{loss, recon, kl_z, kl_S, kl_tau, x_hat, z, mu, log_var}`. The `L_z` key present in v1 is not returned (PR #35: no runtime Laplacian is exposed).

Model factory. `from_config(cfg, E)` dispatches on `cfg['model']['version']`: version 1 builds `WiringAutoencoder`; version 2 builds `WiringAutoencoderV2`.

6 Spectral Artefact and Associative Memory

6.1 Spectral Artefact

After training to convergence, the spectral artefact $\mathcal{A}(\mathcal{I})$ is extracted by evaluating the model at the prior mean $\mathbf{z} = \mathbf{0}$:

$$\mathcal{A}(\mathcal{I}) = \left\{ \hat{\mathbf{W}} = \mathbf{U}_q \text{diag}(\hat{\boldsymbol{\omega}}) \hat{\mathbf{S}}, \quad \hat{\boldsymbol{\omega}} = (\hat{\omega}_1, \dots, \hat{\omega}_q), \quad S_{\mathcal{I}} = \sum_{k=1}^q \hat{\omega}_k \hat{\mathbf{w}}_k \hat{\mathbf{w}}_k^\top \right\}, \quad (20)$$

where $\hat{\mathbf{w}}_k$ is the k -th column of $\hat{\mathbf{W}}$. Note that (20) uses $d_\theta(\hat{\mathbf{w}}_k) \approx \hat{\mathbf{w}}_k$ (linear decoder approximation for $S_{\mathcal{I}}$); a full nonlinear value function $d_\theta(\hat{\mathbf{w}}_k)$ is left as future work.

Table 2: Components of the spectral artefact $\mathcal{A}(\mathcal{I})$.

Component	Symbol	Meaning
Posterior loading basis	$\hat{\mathbf{W}} = (1, d, q)$	Mean loading matrix in Laplacian eigenbasis
Posterior mode weights	$\hat{\boldsymbol{\omega}} \in \mathbb{R}^q$	Per-mode activity under τ -mode prior
Associative memory matrix	$S_{\mathcal{I}} \in \mathbb{R}^{d \times d}$	Outer-product Hopfield key matrix

6.2 SpectralAssociativeMemory

`SpectralAssociativeMemory` (`wae/spectral_memory.py`) wraps $S_{\mathcal{I}}$ as a pre-built Hopfield / linear associative memory. For query $\mathbf{q} \in \mathbb{R}^{d_k}$, retrieval proceeds via:

$$\mathbf{o} = \text{softmax}\left(\beta_H S_{\mathcal{I}}^\top \mathbf{q}\right) S_{\mathcal{I}}, \quad (21)$$

where β_H is the inverse temperature controlling retrieval sharpness. The approximately-orthonormal keys $\hat{\mathbf{w}}_k$ (columns of $\hat{\mathbf{W}}$ are linear combinations of eigenvectors, orthonormal when $\mathbf{S} \approx \mathbf{I}$) maximise retrieval SNR [Ramsauer et al., 2021]: $\text{SNR}^{-1} \approx N_{\text{stored}}/d_k$.

Online delta-rule updates. New associations can be written without overwriting spectral structure:

$$S_{\mathcal{I}} \leftarrow S_{\mathcal{I}} + \frac{1}{B}(\mathbf{v} - S_{\mathcal{I}}\mathbf{k})\mathbf{k}^\top, \quad (22)$$

where $(\mathbf{k}, \mathbf{v})$ are new key-value pairs.

7 Two-Phase Architecture

WAE v2 cleanly separates spectral learning from online inference.

No runtime eigensolves. Because $(\mathbf{U}_q, \boldsymbol{\Lambda}_q)$ are frozen constants in Phase 1, *no Laplacian is computed or inverted at each training step*. All $O(N^3)$ eigendecomposition cost is paid once in offline preprocessing.

Algorithm 1 WAE v2 Two-Phase Procedure

Require: ArrowSpace index $\mathcal{I}$, dataset D , hyperparameters

```
1: // Phase 1 — Offline
2: Compute eigendecomposition:  $(\mathbf{U}_q, \mathbf{\Lambda}_q) \leftarrow \text{eigh}(L(\mathcal{I}))$  ▷ once;  $O(N^3)$ 
3: Freeze  $(\mathbf{U}_q, \mathbf{\Lambda}_q)$ 
4: for each training step do
5:    $\mathbf{z}, \boldsymbol{\mu}, \log \mathbf{s}, \log \mathbf{a}, \log \mathbf{b} \leftarrow \text{WiringEncoderV2}(\mathbf{x})$ 
6:    $\mathbf{W}, \boldsymbol{\omega}, \mathbf{S}, L(\mathbf{z}) \leftarrow \text{SpectralLoadingDecoder}(\mathbf{z}, \mathbf{U}_q, L_{\text{base}})$ 
7:    $\hat{\mathbf{x}} \leftarrow \text{DiffusionDecoder}(L(\mathbf{z}), \mathbf{E})$ 
8:   Compute  $\mathcal{L}_{\text{WAE-v2}}$  (eq. (11)) and update  $\phi, \theta$ 
9: end for
10:  $\mathcal{A}(\mathcal{I}) \leftarrow \text{extract\_spectral\_artefact}(\mathbf{U}_q, \mathbf{\Lambda}_q)$ 
11:  $S_{\mathcal{I}} \leftarrow \text{SpectralAssociativeMemory}(\mathcal{A}(\mathcal{I}))$ 
12: // Phase 2 — Online
13: Initialise transformer FFN / cross-attention from  $S_{\mathcal{I}}$ 
14: for each inference query  $\mathbf{q}$  do
15:   Retrieve via eq. (21); optionally update via eq. (22)
16: end for
```

ELBO Bayes factor for index selection. Training WAE v2 on competing indices $\mathcal{I}_1, \mathcal{I}_2$ yields ELBO estimates $\mathcal{L}(\mathcal{I}_1), \mathcal{L}(\mathcal{I}_2)$. The Bayes factor

$$\text{BF}_{12} = \exp(\mathcal{L}(\mathcal{I}_1) - \mathcal{L}(\mathcal{I}_2)) \quad (23)$$

provides a principled comparison of index constructions: an index whose spectral geometry explains the data covariance better will achieve a higher ELBO [Lotfi et al., 2022].

8 Stability Analysis

WAE v2 inherits the full v1 stability hierarchy from `docs/v2/04-stability.md`. Two additional conditions apply:

Sample-graph Laplacian feedback. When the Laplacian-precision latent KL (design option) is enabled, L_s is built from the batch of latent means $\boldsymbol{\mu}_z$, creating a feedback loop. Prevention: apply stop-gradient on L_s construction; or use a frozen sample Laplacian refreshed every T_{refresh} epochs.

Spectral-basis KL conditioning. If $\lambda_1 \approx 0$ (Fiedler vector near zero), the prior on the first mode becomes nearly flat, increasing gradient variance. Apply a floor $\lambda_k \leftarrow \max(\lambda_k, \epsilon)$ with $\epsilon = 10^{-3}$ in all KL computations (already implemented in `spectral.py` as `eigvals.clamp(min=1e-6)`).

Mode weight collapse. High-frequency modes may collapse to $\omega_k \rightarrow 0$ if $\tau\lambda_k$ is large. Monitor active mode count $|\{k : \mathbb{E}[\omega_k] > \delta\}|$; enforce a soft floor on the Gamma shape $a_k \geq a_{\min}$.

9 Benchmark Metrics

`evaluate_v2()` in `wae/metrics.py` computes the eight metrics in Table 3. The ELBO Bayes factor enables ArrowSpace index selection from data alone.

Table 3: WAE v2 benchmark metrics.

Metric	Computation	Measures
Reconstruction MSE	$\mathbb{E}_{\text{test}}[\ \mathbf{x} - \hat{\mathbf{x}}\ ^2]$	Quality of wiring-path reconstruction
KL spectral basis	$\text{KL}(q(\mathbf{S})\ p(\mathbf{S} \mid \mathcal{I}))$	Spectral alignment with index
τ -mode KL	$\text{KL}(q(\boldsymbol{\omega})\ p(\boldsymbol{\omega} \mid \tau, \boldsymbol{\Lambda}))$	Frequency band selection
Active mode count	$\sum_k \mathbf{1}[\mathbb{E}[\omega_k] > 0.01]$	Effective spectral bandwidth
Memory retrieval SNR	d_k/N_{stored} (key orthogonality)	Associative memory quality
ELBO Bayes factor	$\exp(\mathcal{L}(\mathcal{I}_1) - \mathcal{L}(\mathcal{I}_2))$	ArrowSpace index comparison
Linear probe accuracy	Logistic regression on frozen $\boldsymbol{\mu}$	Discriminative latent quality
Spectral entropy $H(\boldsymbol{\Lambda})$	$H(\text{normalised eigenvalues of } L(\mathbf{z}))$	Wiring diversity

10 Algorithm Tracks

The WAE v2 framework supports six implementation tracks. The recommended sequence is $6 \rightarrow 1 \rightarrow 4 \rightarrow 3$.

- **Option 1 — Deterministic AE.** Replace J_{freq} with the soft τ -mode KL; add `SpectralLoadingDecoder` as ablation. Backward-compatible: $\beta \rightarrow 0$ recovers the deterministic limit.
- **Option 3 — Latent diffusion.** WAE v2 provides the Stage 1 encoder; the spectral noise schedule $a_\tau^{(k)} = \exp(-\tau\lambda_k t)$ matches the $\boldsymbol{\Lambda}_q^{-1}$ latent prior.
- **Option 4 — Variational Laplace AE.** WAE v2 is the full MC-ELBO version; Laplace covariance $(H_{\text{recon}} + \boldsymbol{\Lambda}_m)^{-1}$ is the deterministic approximation to the spectral-precision posterior.
- **Option 6 — Classifier/Reasoner.** The signed density matrix $\varrho_t = \varrho_t^+ - \varrho_t^-$ from the VDT backbone can be used to initialise $S_{\mathcal{I}}$, grounding associative memory in reasoning-time spectral structure.

11 Relationship to Prior Work

WAE v2 integrates ideas from several lines of work:

Graph VAEs. Kipf and Welling [2016] encode graph structure into a VAE but do not impose Laplacian spectral priors on latent codes or loading matrices. WAE v2 makes the spectral structure explicit and variational.

Spectral graph convolution. Graph spectral methods [Defferrard et al., 2016] use Laplacian eigenvectors as filter bases; WAE v2 places this basis *in the generative model* as the loading matrix parametrisation.

Laplacian priors. Rao et al. [2008] and Slawski and Hein [2015] use Laplacian quadratic-form priors for Bayesian variable selection; WAE v2 applies this to both the loading matrix $\mathbf{S}$ and (optionally) the latent code $\mathbf{z}$.

Hopfield / associative memory. Ramsauer et al. [2021] extended Hopfield networks to continuous attractors; Geva et al. [2021] showed that transformer FFN layers are key-value memories. WAE v2 unifies these by seeding the memory with spectral artefacts from a generative model.

VDT backbone. The encoder uses the Vibrational Discrete Transformer from Moriondo [2026b], which implements a discrete damped wave equation in the graph spectral domain as the recurrent state update.

12 Discussion and Open Problems

Posterior variance head for $\mathbf{S}$. The current implementation approximates $\log \text{var}_{S,kj} \approx \log(S_{kj}^2 + \epsilon)$. Adding a dedicated variance head in `SpectralLoadingDecoder` would give a proper amortised Gaussian posterior for $\mathbf{S}$ and is a prioritised open task.

Laplacian-precision latent KL. PR #35 removed this term from the default ELBO after finding that the feedback loop between $\boldsymbol{\mu}_z$ and the dynamically-constructed L_s required careful stop-gradient discipline and added training instability. A frozen sample Laplacian (refreshed every T epochs) may resolve this; experiments are ongoing.

Nonlinear value function. The current spectral artefact uses a linear decoder approximation $d_\theta(\hat{\mathbf{w}}_k) \approx \hat{\mathbf{w}}_k$ for constructing $S_{\mathcal{I}}$. Using the full `DiffusionDecoder` response at each spectral direction would produce a proper key-value Hopfield structure and is left as future work.

Iterative eigensolver. For large graphs ($N \gg 10^3$), truncated LOBPCG or ARPACK can replace full `eigh` in the offline phase, reducing the preprocessing cost from $O(N^3)$ to $O(N \cdot q)$.

13 Conclusion

WAE v2 upgrades the Wiring Autoencoder from a heuristically regularised generative model to a principled Bayesian VAE in which the latent space and loading matrix are explicitly shaped by Laplacian spectral geometry provided by an `ArrowSpace` index. The three-term ELBO (Eq. 11) replaces the hard J_{freq} penalty with interpretable variational KL terms; the two-phase offline/online architecture eliminates runtime eigensolves; and the spectral artefact $\mathcal{A}(\mathcal{I})$ provides a principled initialiser for transformer associative memory. ELBO Bayes factors make `ArrowSpace` index selection a first-class modelling decision.

References

- F. R. K. Chung. *Spectral Graph Theory*. American Mathematical Society, 1997.
- M. Defferrard, X. Bresson, and P. Vandergheynst. Convolutional neural networks on graphs with fast localised spectral filtering. In *Advances in Neural Information Processing Systems*, 2016.
- M. Figurnov, S. Mohamed, and A. Mnih. Implicit reparameterization gradients. In *Advances in Neural Information Processing Systems*, 2018.
- M. Geva, R. Schuster, J. Berant, and O. Levy. Transformer feed-forward layers are key-value memories. *arXiv preprint arXiv:2012.14913*, 2021.
- T. N. Kipf and M. Welling. Variational graph auto-encoders. In *NIPS Workshop on Bayesian Deep Learning*, 2016.
- S. Lotfi, P. Izmailov, G. Benton, M. Goldblum, and A. G. Wilson. Bayesian model selection, the marginal likelihood, and generalisation. In *International Conference on Machine Learning*, 2022.

- T. P. Minka. Automatic choice of dimensionality for PCA. In *Advances in Neural Information Processing Systems*, volume 13, 2001.
- Moriondo. Spectral-PPCA VAE: A Laplacian-prior generative framework for spectral associative memory in transformers. Technical report, tuned-org-uk, 2026a. https://github.com/tuned-org-uk/vibrational-deduction-transformer/blob/main/docs/v2/00-design-A_v2%20--%20Spectral-PPCA.md.
- Moriondo. Vibrational discrete transformer (VDT). Technical report, tuned-org-uk, 2026b. <https://github.com/tuned-org-uk/vibrational-deduction-transformer/blob/main/wae/vdt.py>.
- H. Ramsauer, B. Schäfl, J. Lehner, P. Seidl, M. Widrich, T. Adler, L. Gruber, M. Holzleitner, M. Pavlović, G. K. Sandve, et al. Hopfield networks is all you need. In *International Conference on Learning Representations*, 2021.
- A. Rao, A. Doufas, and L. Li. Bayesian regularization via graph Laplacian. *Bayesian Analysis*, 3(2):435–462, 2008.
- M. Slawski and M. Hein. A graph Laplacian prior for Bayesian variable selection and grouping. *Computational Statistics and Data Analysis*, 2015.
- M. E. Tipping and C. M. Bishop. Probabilistic principal component analysis. *Journal of the Royal Statistical Society: Series B*, 61(3):611–622, 1999.